

UNIVERSIDAD TECNOLÓGICA DEL PERÚ

FACULTAD DE INGENIERÍA DE SISTEMAS E INFORMÁTICA

CURSO INTEGRADOR II: SOFTWARE (100000S12F)

GUÍA FORMAL DE PREGUNTAS Y RESPUESTAS CRÍTICAS PARA SUSTENTACIÓN ANTE EL JURADO

PROYECTO: SISTEMA WEB PWA DE GESTIÓN Y TOMA DE PEDIDOS MULTICANAL PARA LEOFIT

1. CATEGORÍA: ANÁLISIS DE NEGOCIO, PROCESOS Y RÚBRICA APF1

Pregunta 1.1: ¿Por qué la empresa seleccionada califica como un caso de estudio válido para el curso?

Fundamentación Técnica y de Negocio:

LeoFit cumple estrictamente con los criterios de la rúbrica al ser una microempresa peruana debidamente constituida, con operación comercial activa y una problemática real demostrable: la gestión manual y no estandarizada de pedidos por WhatsApp que genera demoras de 25 minutos por orden, sobreventas y extravío de registros. Además, se cuenta con acceso irrestricto al stakeholder (Víctor Raúl Cárdenas), garantizando la validación continua de prototipos e incrementos de software.

Pregunta 1.2: ¿En qué se diferencia el modelo AS-IS del modelo propuesto TO-BE?

Fundamentación Técnica y de Negocio:

El modelo AS-IS es secuencial, manual y vulnerable, con 8 pasos donde la verificación de stock se hace visualmente y las anotaciones en papel. El modelo TO-BE digitaliza el proceso mediante una Progressive Web App (PWA): el cliente consulta un catálogo interactivo con stock sincronizado, el sistema valida y descuenta existencias automáticamente, calcula el flete y genera una orden formal con ID único. El administrador gestiona la preparación y el despacho mediante un tablero con cambio de estados en un solo clic, reduciendo el ciclo de pedido a menos de 3 minutos.

2. CATEGORÍA: GESTIÓN ÁGIL, SCRUM Y PLANIFICACIÓN

Pregunta 2.1: ¿Cómo determinaron la velocidad del equipo y la duración de los Sprints?

Fundamentación Técnica y de Negocio:

Se estableció un esquema de Sprints de 2 semanas calendario con una capacidad estimada de 35 Story Points (SP) por iteración. Las historias de usuario fueron estimadas mediante la secuencia Fibonacci (1, 2, 3, 5, 8, 13) y priorizadas con la metodología MoSCoW. Cada historia cuenta con criterios de aceptación en estándar Gherkin (*Given-When-Then*), y el flujo de trabajo se controla en un tablero Kanban con límites estrictos de trabajo en progreso (WIP Limits) en GitHub Projects.

Pregunta 2.2: ¿Cuál es la política de Definition of Done (DoD) acordada por el equipo?

Fundamentación Técnica y de Negocio:

Para que una tarea se considere concluida (**Done**), debe cumplir cuatro criterios rigurosos: (1) Código implementado en TypeScript estricto sin advertencias de linters; (2) Pruebas unitarias aprobadas con Vitest garantizando $\geq 80\%$ de cobertura; (3) Revisión por pares mediante Pull Request en GitHub; y (4) Despliegue automatizado exitoso en el ambiente de producción mediante GitHub Actions.

3. CATEGORÍA: ARQUITECTURA DE SOFTWARE Y TECNOLOGÍA FRONT-END

Pregunta 3.1: ¿Por qué seleccionaron una Progressive Web App (PWA) con React y Vite en lugar de una aplicación móvil nativa o WordPress?

Fundamentación Técnica y de Negocio:

- **Frente a Apps Nativas:** Una PWA no requiere instalación desde tiendas propietarias (Play Store / App Store), eliminando la fricción de descarga para el cliente y reduciendo drásticamente los costos de desarrollo multi-plataforma (un solo código fuente para móvil y escritorio).
- **Frente a WordPress/WooCommerce:** React + Vite proporciona una arquitectura desacoplada, carga instantánea (< 0.5 s), menor consumo de memoria y control total sobre el pipeline de optimización WPO.
- **Uso de TypeScript y Tailwind:** Garantiza tipado estricto en tiempo de compilación, previniendo errores en tiempo de ejecución, y un bundle de estilos ultraligero depurado con PurgeCSS.

4. CATEGORÍA: ASEGURAMIENTO DE LA CALIDAD (QA) Y PRUEBAS

Pregunta 4.1: ¿Qué estrategia de pruebas se implementó para garantizar la fiabilidad del sistema?

Fundamentación Técnica y de Negocio:

Se implementó una suite de pruebas unitarias automatizadas con **Vitest** en la capa de datos y lógica de negocio (`frontend/src/__tests__/mockData.test.ts`). Las pruebas validan la integridad de precios, existencias no negativas, pertenencia de categorías y normalización de estados de los pedidos. La suite se ejecuta de forma automática en el pipeline de Integración Continua (`.github/workflows/security-scan.yml`) ante cada `push` y `pull request` a la rama `main`.

5. CATEGORÍA: OPTIMIZACIÓN WEB (WPO) Y NIVELES DE SERVICIO (SLA/SLO)

Pregunta 5.1: ¿Cuáles fueron los resultados cuantitativos de las estrategias WPO implementadas?

Fundamentación Técnica y de Negocio:

Se implementaron cinco estrategias técnicas: Tree-shaking, minificación con esbuild, code-splitting modular, memoización React y carga asíncrona de fuentes. Los resultados medidos con Google Lighthouse evidencian:

- Incremento de la puntuación Lighthouse de 68/100 a **98/100** (+44.1%).
- Reducción del First Contentful Paint (FCP) de 2.4 s a **0.3 s** (-87.5%).
- Reducción del tamaño del bundle JavaScript gzipped de 450.0 kB a **74.49 kB** (-83.4%).
- Tiempo de compilación en producción reducido a **0.73 segundos**.

Pregunta 5.2: ¿Cuáles son los compromisos de SLA y SLO establecidos para LeoFit?

Fundamentación Técnica y de Negocio:

Se estableció un SLA de disponibilidad mensual $\geq 99.5\%$ respaldado por CDN Serverless en el Edge, un SLO de latencia de renderizado P95 < 500 ms, un tiempo máximo de recuperación ante fallos (RTO) < 2 horas y un punto de recuperación de datos (RPO) < 15 minutos mediante persistencia local y sincronización en la nube.